

Roleva — Build to Deployment Checklist

Living tracker. Tick items as you complete them.

Rule: **do not start a phase until the previous phase's GATE passes.**

WHERE YOU ARE RIGHT NOW

- Requirements defined
- Ambiguities resolved (A1–A8, B1–B8, C1–C12)
- Architecture designed
- Tech stack chosen
- Scoring system designed
- Roadmap defined
- ← **YOU ARE HERE. Next: Phase -1, external accounts.**

Progress: 6 / 214 tasks (~3%) — planning complete, zero code written.

PROGRESS DASHBOARD

Phase	Name	Tasks	Done	Status
-1	Accounts & Prerequisites	14	0	Not started
0	Foundation	23	0	Not started
1	Parsing Pipeline	20	0	Not started
2	JD & Matching	18	0	Not started
3	Scoring & ATS	19	0	Not started
4	Advice Engine	9	0	Not started
5	API & Orchestration	16	0	Not started
6	Design Exploration	12	0	Not started
7	Frontend Build	32	0	Not started
8	Sharing & Percentiles	11	0	Not started
9	Hardening	18	0	Not started
10	Deployment	14	0	Not started
11	Launch	10	0	Not started
TOTAL		216	0	

PHASE -1: ACCOUNTS & PREREQUISITES

No code. Just accounts and keys. Do this in one sitting.

- 1.1 GitHub repo `roleva` created (private for now)
- 1.2 Node.js 20+ installed, `node -v` verified

- 1.3 pnpm installed, `pnpm -v` verified
- 1.4 Python 3.12 installed, `python --version` verified
- 1.5 Docker Desktop installed and running
- 1.6 Google AI Studio account created
- 1.7 Gemini API key generated and stored safely
- 1.8 **Gemini free-tier limits recorded** (RPM / RPD / embedding availability)
- 1.9 Supabase account + project `roleva-dev` created
- 1.10 Supabase project `roleva-prod` created
- 1.11 Vercel account created, GitHub connected
- 1.12 Render account created, GitHub connected
- 1.13 Sentry account created (free tier)
- 1.14 `.env.example` written; real `.env` gitignored

GATE -1

All keys work — a one-off script successfully calls Gemini and returns text

PHASE 0: FOUNDATION

The contract layer. Get this right or everything downstream churns.

Repo & tooling

- 0.1 Monorepo scaffold: `apps/web` , `apps/api` , `packages/contracts` , `docs`
- 0.2 pnpm workspace config
- 0.3 Python package + `pyproject.toml` (ruff, mypy, pytest)
- 0.4 `.gitignore` , `.editorconfig` , `README.md`
- 0.5 `docker-compose.yml` for local API
- 0.6 `docs/ARCHITECTURE.md` committed

Database & auth

- 0.7 Supabase migration: `profiles` , `resumes` , `job_targets` , `analyses`
- 0.8 Supabase migration: `share_links` , `score_samples` , `cohort_stats` , `rate_limits` , `llm_usage`
- 0.9 **RLS policies enabled and tested on every user table**
- 0.10 Supabase Auth configured (email + Google OAuth, verification ON)

Skeletons

- 0.11 FastAPI app boots, `/health` returns 200
- 0.12 JWT verification middleware (validates Supabase token)
- 0.13 Next.js app boots, Tailwind v4 + `tokens.css` in place
- 0.14 Structured logging with **no content logged**

The contract

- 0.15 `ResumeDocument` schema — **builder-ready, versioned**
- 0.16 `Requirement`, `Evidence`, `AtsFinding`, `ScoreReport`, `AnalysisReport` schemas
- 0.17 OpenAPI → TypeScript type generation working

LLM & privacy layer

- 0.18 LLM provider adapter interface + Gemini implementation
- 0.19 Token-bucket rate limiter + `llm_usage` tracking
- 0.20 **PII redactor + restorer, with unit tests**
- 0.21 CI: lint + typecheck + pytest + vitest on every push

Test corpus (do not skip)

- 0.22 40+ anonymized resumes collected (Word, Canva, LaTeX, Google Docs, 1-col, 2-col)
- 0.23 15+ job descriptions collected (SDE, data, product, design; intern + entry)

GATE 0

- CI green
- A user can sign up, verify email, and log in
- RLS proven: user A cannot read user B's row
- PII redactor round-trips text with zero loss

PHASE 1: PARSING PIPELINE

Highest-risk subsystem. Everything downstream inherits its errors.

- 1.1 Upload validation: MIME + magic bytes
- 1.2 Upload validation: size (8MB), pages (10), encryption, corruption
- 1.3 Scanned-PDF detection (text density) + helpful rejection message
- 1.4 English-language detection
- 1.5 PyMuPDF extraction: blocks, bboxes, fonts, sizes, **colors**
- 1.6 Absolute character offset tracking (span foundation)
- 1.7 Column detection via x-coordinate clustering
- 1.8 Reading-order reconstruction
- 1.9 Normalizer: unicode, ligatures, bullets, de-hyphenation
- 1.10 **Offset-preserving** normalization map
- 1.11 Repeated header/footer stripping
- 1.12 Section-header lexicon (~120 variants)
- 1.13 Multi-signal heading scorer (font, caps, bold, length, gap)
- 1.14 Sectionizer producing sections with spans
- 1.15 Contact-block extraction (feeds the PII redactor)
- 1.16 Resume structuring LLM call (redacted input, flat Gemini-safe schema)
- 1.17 Pydantic validation + one repair retry + deterministic fallback

1.18 **Span verification — drop any item not found verbatim in the text**

1.19 Deterministic date parsing + durations + gap computation

1.20 `parse_confidence` scoring + degradation behavior

GATE 1

Full corpus runs without crashing

≥90% of corpus parses at confidence ≥0.7 (manually verified)

Golden snapshots committed for all corpus resumes

Two-column resumes parse in correct reading order

PHASE 2: JD & MATCHING

2.1 JD length validation (<200 reject, <600 warn)

2.2 JD cleaner (strip EEO, benefits, salary, "about us")

2.3 Requirement extraction LLM call

2.4 Role-family + seniority classification (same call, no extra request)

2.5 Deterministic priority cue rules (must / strong / nice override layer)

2.6 Quantifier regex ("3+ years")

2.7 Skill taxonomy seeded (~800 canonical entries + aliases)

2.8 Alias resolver

2.9 Requirement deduplication

2.10 Tier 1: canonical/exact matching

2.11 Tier 2: fuzzy matching (rapidfuzz, ≥88)

2.12 Batched embedding call + content-hash cache

2.13 Tier 3: cosine banding (≥0.82 accept, 0.62–0.82 ambiguous, <0.62 reject)

2.14 Tier 4: **single batched adjudication call** for all ambiguous items

2.15 **Evidence strength by location** (bullet 1.0 / project 0.85 / skills-only 0.5 / adjacent 0.4)

2.16 Years-of-experience resolution from parsed dates

2.17 Evidence span verification

2.18 Unmatched-resume-content detection (what you have that the JD doesn't want)

GATE 2

≥85% agreement with your manual judgment on the 20-pair calibration set

Total LLM calls per analysis measured and ≤ 6

Zero unverifiable evidence spans reach output

PHASE 3: SCORING & ATS

3.1 ATS rule registry framework

3.2 Check: multi-column layout (–15)

3.3 Check: tables (–12), text-as-image (–25)

- 3.4 Check: header/footer content (–10)
- 3.5 Check: missing/nonstandard sections (–12 / –8)
- 3.6 Check: missing contact fields (–10 each)
- 3.7 Check: unparseable dates (–6), exotic fonts (–5)
- 3.8 Check: **hidden text detection** (–20 + warning)
- 3.9 Check: excessive length (–8), filename (–2), special chars (–4)
- 3.10 Deterministic writing metrics (quantification, action verbs, bullet length, weak phrases, passive voice, pronouns, tense, repetition, density)
- 3.11 Basic grammar/spelling flagging (B7)
- 3.12 Anchored quality rubric LLM call (all sections, one call)
- 3.13 `rubric.yaml` with versioned weights
- 3.14 **Pure scoring engine** (zero I/O, zero LLM) — Job Match, ATS, Quality, Overall
- 3.15 Must-have gate (caps at 55 / 72)
- 3.16 Explanation objects (components → contributions → evidence refs)
- 3.17 Expected-band reference table (Track 2 relative scoring)
- 3.18 `score_samples` write path (unlinkable — no user_id)
- 3.19 Golden score regression tests in CI

GATE 3

- Same inputs produce **identical** scores across 5 runs
- Manual calibration: system's rank order matches yours on 20 pairs
- Every score expands to its components; every component links to evidence
- No number anywhere in the system originates from an LLM

PHASE 4: ADVICE ENGINE

- 4.1 Weak-bullet identification (deterministic selection)
- 4.2 Merged advice LLM call (suggestions + recommendations + summary)
- 4.3 **Grounding validator** — reject rewrites introducing new facts
- 4.4 Regenerate-once-then-drop policy for ungrounded output
- 4.5 Recommendation generation from gaps + ATS findings
- 4.6 **Impact ranking** — recompute score delta, show projected gain
- 4.7 Deduplication
- 4.8 Hard cap at 5–7 recommendations
- 4.9 Summary paragraph built from computed facts only

GATE 4

- Spot-check 10 analyses: every suggestion is specific to that resume, not generic
 - Zero fabricated numbers/employers/technologies in suggested bullets
-

PHASE 5: API & ORCHESTRATION

- 5.1 Pipeline orchestrator with per-stage isolation
- 5.2 Parallelize independent stages
- 5.3 SSE progress event contract
- 5.4 SSE emitter + partial-result delivery
- 5.5 `POST /analyze` (multipart)
- 5.6 `GET /stream/{id}`
- 5.7 `GET /analyses`, `GET /analyses/{id}`, `DELETE /analyses/{id}`
- 5.8 Persistence to Supabase (resume, job_target, analysis)
- 5.9 Content-hash caching (resume + JD) — skip redundant LLM calls
- 5.10 Rate limiting (per-user 5/day, per-IP 20/hr)
- 5.11 Global daily LLM budget guard + graceful "capacity reached"
- 5.12 Exception → user-facing message mapping (every failure mode)
- 5.13 **Warm-up endpoint** + frontend ping strategy
- 5.14 GitHub Actions cron ping (keeps Render + Supabase awake)
- 5.15 Sentry + per-stage timing telemetry
- 5.16 Integration tests across full corpus

GATE 5

- End-to-end analysis works against a real Gemini key
- Warm p95 latency < 25s
- Cold start handled with an honest UI state, not a hang
- Every error path returns a specific, actionable message

PHASE 6: DESIGN EXPLORATION (*approval gate*)

- 6.1 Reference research — what to avoid, what to aim for
- 6.2 **Direction A** — concept, typography, color, layout, motion, sample report screen
- 6.3 **Direction B** — same deliverables
- 6.4 **Direction C** — same deliverables
- 6.5 Present all three with trade-offs
- 6.6 **YOU CHOOSE ONE**
- 6.7 Design token system built (color, type, space, radius, motion)
- 6.8 Score-visualization language designed (custom SVG)
- 6.9 Requirement-map visualization designed
- 6.10 Evidence-linking interaction model designed
- 6.11 Motion system (durations, easings, choreography rules)
- 6.12 Dark/light mode decision + palette locked

GATE 6

- One direction chosen and documented in `docs/DESIGN.md`

Tokens implemented and rendering

Palette passes AA contrast before any screen is built

PHASE 7: FRONTEND BUILD

Primitives

- 7.1 Button, Field, Input, Textarea
- 7.2 Dialog, Tooltip, Popover, Disclosure (Radix-based)
- 7.3 Toast / notification system
- 7.4 Loading + skeleton states

Auth

- 7.5 Sign-up screen
- 7.6 Sign-in screen
- 7.7 Email verification flow
- 7.8 Password reset
- 7.9 Google OAuth button
- 7.10 Protected route handling

Core flow

- 7.11 Landing page (with cached demo analysis, no LLM cost)
- 7.12 Upload: dropzone + file picker + mobile
- 7.13 Client-side preflight validation + specific errors
- 7.14 JD input with length meter + quality hint
- 7.15 **Analysis progress screen** (real stage names + cold-start state)
- 7.16 SSE client + progressive report hydration

Report

- 7.17 Verdict / competitiveness band
- 7.18 Four score visualizations (custom SVG)
- 7.19 Expected-band relative display
- 7.20 Score expansion → components → contributions
- 7.21 Requirement map (JD ↔ resume evidence)
- 7.22 Skill ledger: matched / partial / missing (must-haves separated)
- 7.23 **Evidence viewer** — click any claim, see the exact resume line
- 7.24 ATS checklist with location + impact + fix
- 7.25 Section-by-section feedback
- 7.26 Bullet improvement panel + copy-to-clipboard
- 7.27 Ranked recommendations with projected score gain

Account surfaces

- 7.28 History dashboard (past analyses)
- 7.29 Settings: data export, delete analysis, delete account
- 7.30 Quota display ("3 of 5 analyses left today")

Quality passes

- 7.31 Responsive pass (375px → 1920px), every screen
- 7.32 Accessibility pass: keyboard nav, screen reader, focus, **no color-only encoding**

GATE 7

- Full flow usable on a phone
 - Lighthouse: Performance ≥85, Accessibility ≥95
 - `prefers-reduced-motion` respected everywhere
 - Any number in the report traces to a highlighted resume line
-

PHASE 8: SHARING & PERCENTILES

- 8.1 Share token generation (32-byte random)
- 8.2 Visibility mode: scores only
- 8.3 Visibility mode: full redacted (default)
- 8.4 Visibility mode: full identified
- 8.5 Expiry (default 7d, max 30d)
- 8.6 Revocation + view counter
- 8.7 Public share view page
- 8.8 `noindex, nofollow` + `X-Robots-Tag` verified
- 8.9 Share-link management UI in settings
- 8.10 Cohort aggregation job → `cohort_stats`
- 8.11 **Percentile display gated at $N \geq 30$** , with sample-size label

GATE 8

- Redaction verified server-side (not just hidden in CSS)
 - Revoked link returns 404 immediately
 - Percentile does not render below $N=30$
-

PHASE 9: HARDENING

Security

- 9.1 Upload attack surface tested (zip bomb, polyglot, malformed)
- 9.2 RLS re-verified with a cross-user test suite
- 9.3 IDOR test on analyses + share tokens

- 9.4 CSP, CORS, HSTS, security headers configured
- 9.5 Secrets audit — no key reachable from the browser
- 9.6 Dependency vulnerability scan
- 9.7 **Prompt-injection test resumes** (hidden text, instruction text)
- 9.8 **PII redaction audit — capture outbound payloads, confirm nothing un-redacted**

Testing

- 9.9 Playwright E2E: happy path
- 9.10 Playwright E2E: every error path
- 9.11 Auth flow E2E
- 9.12 Cold-start behavior verified on real Render free tier
- 9.13 Load test to find actual concurrency ceiling
- 9.14 Quota exhaustion behavior verified

Legal & docs

- 9.15 Privacy policy (Gemini processing + redaction + retention)
- 9.16 Terms of service + "not employment advice" disclaimer
- 9.17 **Public scoring methodology page** (publish the rubric)
- 9.18 Consent checkbox at signup

GATE 9

- Zero high-severity security findings
- All E2E tests green
- Legal pages live and linked

PHASE 10: DEPLOYMENT

- 10.1 Supabase prod project configured + migrations applied
- 10.2 Prod RLS policies verified (again, on prod)
- 10.3 Dockerfile for API, image size minimized
- 10.4 Render service created, env vars set
- 10.5 API deploys and `/health` responds in prod
- 10.6 Vercel project connected, env vars set
- 10.7 Frontend deploys and loads
- 10.8 CORS locked to the prod web origin
- 10.9 Supabase Auth redirect URLs set for prod domain
- 10.10 Custom domain (or accept `*.vercel.app`)
- 10.11 HTTPS + HSTS verified
- 10.12 GitHub Actions cron ping pointed at prod
- 10.13 Sentry receiving prod errors
- 10.14 **Full end-to-end analysis completed in production**

GATE 10

- A stranger on a phone can sign up and get a real analysis
 - Cold start recovers gracefully in prod
 - No secrets in the client bundle (verified by inspecting build output)
-

PHASE 11: LAUNCH

- 11.1 10 real testers recruited
 - 11.2 Feedback collected on **advice quality** specifically
 - 11.3 Calibration re-tuned from real data
 - 11.4 Top 3 usability issues fixed
 - 11.5 Daily LLM usage monitored for one week
 - 11.6 Quota levels adjusted to real usage
 - 11.7 Onboarding copy refined
 - 11.8 Empty states + first-run experience polished
 - 11.9 Repo made public / portfolio write-up
 - 11.10 **LAUNCH**
-

SUCCESS DEFINITION

Roleva v1 is done when:

1. A user signs up, uploads a PDF, pastes a JD, and gets a report in under 30 seconds.
 2. Every number in that report can be expanded to show exactly how it was computed.
 3. Every claim links to a highlighted line in their own resume.
 4. Suggestions are specific to their content, never generic.
 5. Their name, email, phone, and address never reached an external API.
 6. Running it twice gives the same scores.
 7. It works on a phone.
 8. It costs ₹0/month to operate.
-

AFTER V1 (do not start early)

- Resume builder (uses the same `ResumeDocument`)
- DOCX support
- OCR for scanned PDFs
- PDF export of reports
- Multi-JD comparison
- Cover letter analysis
- Paid tier